Resuelve los siguientes ejercicios y entrega el código fuente tanto en un fichero .pdf como en ficheros de texto java o la aplicación ejecutable según se pida en el ejercicio, en el .pdf capturas de pantalla (si se piden) y las respuestas a las preguntas que se plantean, si las hay.

EJERCICIOS PADAWAN (OBLIGATORIOS)

EJERCICIO 1. ELEGIR COLECCIÓN ADECUADA A UN PROBLEMA.

En un programa necesitamos leer textos de documentos y contar cuantas veces aparece cada palabra. Mide el tiempo que tarda el programa en realizar esta tarea. Para probarlo, vamos a usar el fichero de texto "quijote_cervantes.txt" y convertiremos todas las palabras a minúscula.

> **Nota**: ten en cuenta que si solamente usas de separadores espacios en blanco y saltos de línea, puedes acabar teniendo palabras acabadas en signos de puntuación (como puntos y comas), por tanto añade estas letras como posibles separadores de palabras.

a) (0.5 PUNTOS): Haz el programa **U08E01A.java** que utilice una colección de tipo lista que sea capaz de almacenar objetos de tipo {palabra:String, contador:long}. . Pasa el código y muestra captura de ejecutar el programa donde se vea cuantas palabras ha contabilizado, cuánto tiempo ha tardado en realizar el proceso y cuál es la palabra de más de 5 letras que más veces aparece.

b) (0.5 PUNTOS). Haz el programa **U08E01B.java** a partir del código del apartado a), pero que no use una lista. Una lista no es la mejor colección para resolver este problema (ya sea la basada en arrays o en enlaces), cuando obtienes una palabra del fichero, tendrás que saber si está en la lista. Si no está, la añades con contador a 1 y si está, le sumas 1 al contador. Pero como ya sabrás, localizar elementos en listas no es eficiente. Piensa en una colección alternativa de las que tenga Java predefinidas que pueda realizar esta tarea de manera eficiente. Me pasas:

- Justificación de la colección escogida (¿Porqué motivo la has seleccionado en lugar de otras?)
- El código del programa.
- Captura de ejecución donde se vea el tiempo empleado en procesar el documento.
- Buscar la misma palabra que devolvió el programa anterior como la más frecuente.

EJERCICIO 2. IMPLEMENTA EL TDA COLA DE ENTEROS.

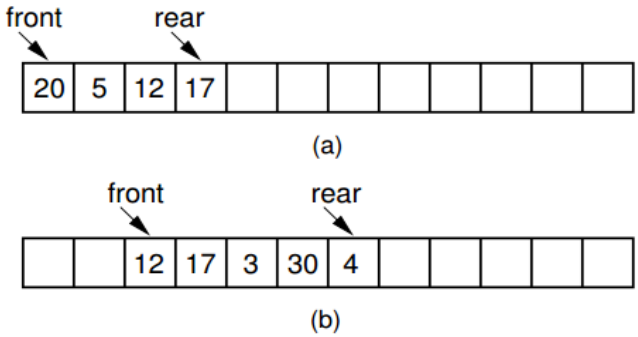
Las colas son una estructura de datos secuencial similar a las listas (un elemento detrás de otro) pero que restringe las posiciones por donde se añaden o eliminan los elementos que la forman. En una cola los elementos se añaden siempre por el final de la cola (operación **encolar(elemento)**) mientras que siempre se sacan por la cabeza (operación **desencolar()**). Es una estructura FIFO "First-In, First-Out". Aquí está una posible interfaz para las Colas:

```
interfaz para Cola
        public void vaciar();
        public boolean encolar(elemento);   // Devuelve true si puede añadir elemento
        public elemento desencolar();       // Devuelve el primer elemento o null
```

```
public elemento primero();          // Devuelve el primer elemento sin quitarlo
public int longitud();       // Devuelve el tamaño de la lista
```

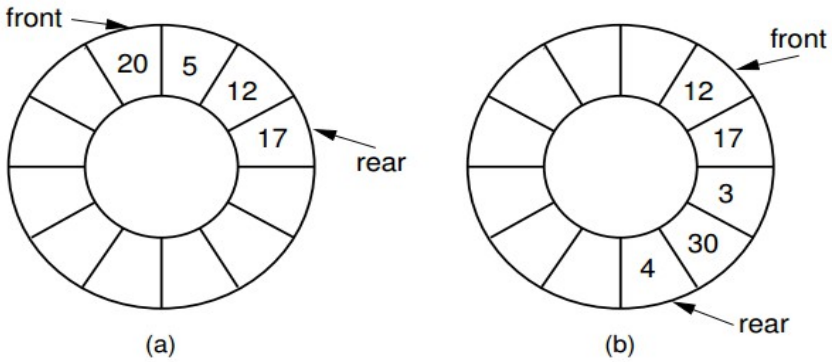
Vamos a usar un array normal de enteros para implementar una cola de números enteros. Si exigimos que los elementos ocupen las primeras posiciones, añadir y eliminar elementos nos obliga a desplazar los elementos que ya haya para no dejar huecos. Las operaciones las hacemos lentas (orden n, donde n es la longitud de la cola, es decir, el número de elementos).

Para hacer las operaciones de **encolar()** y **desencolar()** más eficientes, **encolar()** va a añadir por el final (como una lista) y desencolar() quitará elementos por el principio del array de forma que la cola se hace pequeña. Usando dos valores enteros que nos digan la posición de la cabeza y el final de la cola, podemos acelerar las operaciones. No dejamos huecos en el array y nos ahorramos mover elementos a cambio de actualizar donde está la cabeza y el final de la cola después de cada cambio.



(a) La cola después de añadir los números 20, 5, 12 y 17
(b) La cola tras desencolar 20 y 5 y luego encolar 3, 30 y 4.

Otra mejora para que la cabeza y el final de la cola puedan moverse libremente por todas las posiciones del array consiste en hacerla circular. Esto lo vamos a conseguir actualizando los valores de la cabeza y el final con la operación módulo.



(a) y (b) Las mismas operaciones de antes pero haciendo circular el array.

Nos queda por resolver un problema sutil. **¿Cómo saber si la cola está llena o vacía?** Podríamos pensar que **si fin < cabeza entonces está vacía**. Pero ¿Y si la cola está completamente llena? Tendríamos la misma condición que si estuviese vacía, no podemos distinguir ambos casos. Así que la única solución es ir contando los elementos que tenemos en otra variable o crear un array con una posición más que el máximo de elementos a añadir y así fin < cabeza solo se produce si está vacía. El esqueleto de una implementación basada en

Arrays:

(0.2 PUNTOS) Define la interfaz ColasInt

(0.6 PUNTOS) Define la clase ColaArrayInt que implemente la interfaz. Puedes usar este esqueleto.

```
class ColaArrayInt implements Colas {
        private static final int LONGITUD_POR_DEFECTO = 10;
        private int maxLongitud;        // Tamaño máximo
        private int cabeza;             // Índice del primero de la cola
        private int fin;                // Índice del último de la cola
        private int[] datos;            // Array que almacena elementos

        /** Constructores */
        public Cola() { this(LONGITUD_POR_DEFECTO); }

        public Cola(int max) {
                maxLongitud ←  max + 1;  // Un espacio extra
                fin ← 0;  cabeza ← 1;
                Inicializa el array datos
        }

        public void vaciar() { // poner el final a 0 y la cabeza a 1 }

        /** Añadir a la cola e */
        public void encolar(elemento) {
            precondición: (fin + 2) MODULO maxLongitud  DISTINTO cabeza
                           Si no se cumple, la cola está llena.
            fin = (fin + 1) % maxLongitud;     // Incremento circular
            datos[fin] = e;
        }

        /** Eliminar y devolver el primer elemento (cabeza) */
        public elemento desencolar() {
            precondicion: longitud() DISTINTO 0
                          Sino: "La cola está vacía";
            elemento ← datos[cabeza];
            cabeza ← (cabeza + 1) MODULO  maxLongitud;    // Incremento Circular
            devolver elemento;
        }

        /** @return primer valor */
        public elemento primero() {
            precondición: longitud() DISTINTO 0
                          sino se cumple: "La cola está vacía";
            devolver datos[cabeza];
        }

        /** @return Cantidad de elementos en la cola */
        public int longitud(){
            devolver ((fin + maxLongitud) - cabeza + 1) % maxLongitud;
        }
}
```

(0.1 PUNTOS) Haz un programa que defina una cola de enteros, la llene con 4 números 1,2,3,4. La imprima, saque 1 número, inserte otro y la vuelva a imprimir.

(0.1 PUNTOS) ¿Qué inconviente tiene esta forma de implementar TDA's? (piensa en que ocurriría si más adelante necesitamos una cola de doubles, luego una de Personas, luego una de

Solicitudes, luego una de Alumnos, luego una de Facturas a pagar, otra de ….)

EJERCICIO 3. IMPLEMENTA COLA DE CUALQUIER ELEMENTO.

a) (0.5 PUNTOS) Modifica el código del ejercicio anterior y define una interfaz y una implementación de colas de Object que sean capaces de almacenar cualquier elemento:

i) (0.1 puntos) Define la interfaz ColaObj.
ii) (0.3 puntos) Implementa una clase ColaArrayObj que implemente la interfaz del apartado i) y sea capaz de almacenar cualquier objeto sin utilizar genéricos.
iii) (0.1 puntos) Crea una cola de enteros, una cola de Strings y una cola de Personas, añade dos o tres elementos, las imprimes, desencola uno y las vuelves a imprimir.

Persona = {nombre, edad}

Nota: Cuando declares en la clase e inicialices en el constructor el array de datos, recuerda que debe poder almacenarse cualquier cosa.

¿Qué dos inconvenientes sigue teniendo nuestro código cuando utilicemos esta clase?

b) (0.5 PUNTOS) Modifica el código anterior, define una interfaz equivalente genérica y una clase ColaArray<T> genérica que sea capaz de almacenar cualquier objeto. Crea una cola de enteros, una cola de Strings y una cola de Personas, y una cola sin indicar el genérico donde guardas personas, las imprimes, desencola uno y las vuelves a imprimir.

Persona = {nombre, edad}

EJERCICIO 4. EL MUNDO DE LOS SIMULADORES.

Imagina que trabajas en una empresa consultora. Una cadena de supermercados va a abrir un nuevo local y quiere estudiar las instalaciones que necesita para optimizar gasto/beneficio del nuevo local y saber si la configuración que han pensado dará los resultados que esperan. Han realizado una serie de estimaciones que se resumen así:

- El local abre de 9:00 a 20:00 horas de cada día.
- El número de clientes que entran al local será diferente cada hora (tasaClientes) y de media se espera en cada franja horaria la siguiente distribución (de 9:00 a 10:00 25, …). La siguiente tabla resume las previsiones (puedes definirlas en un array):

09-10: 25	12-13: 65	15-16: 18	18-19: 40
10-11: 40	13-14: 80	16-17: 21	19-20: 60
11-12: 50	14-15: 42	17-18: 32	

- Han pensado en habilitar 2 cajas donde se cobra a los clientes.
- Una vez entra un cliente, tarda cierto tiempo (coge carro/cesta, se pasea por el local buscando los productos que necesita, …) al que llamamos **tBuscar**, antes de dirigirse a la zona de cajas para pagar. Es un campo que tendrá un cliente. Cada cliente tiene otro campo (tQueda) que cuenta los minutos que le faltan para acabar de explorar/pagar. El

valor tBuscar de los clientes sigue una distribución normal de media 10 minutos y desviación típica de 5,8. La cantidad de productos que compran suele ser un múltiplo del tiempo que permanecen en el local (a más tiempo, más productos) y cada cliente tiene el campo **compras** que se calcula como tBuscar * aleatorio, donde aleatorio sigue una distribución uniforme [0, 2).

- Cuando el cliente llega a la zona de cajas (si tQueda = 0), elige la caja con la menor cola esperando a que le cobren. Se tarda en cobrar a cada cliente una media de (2 + producto/4) minutos. Cuando el cliente es el primero de la cola, comienza el proceso de pago, pone tQueda a tPagar y va restando hasta que llegue a 0. En ese momento sale del local.

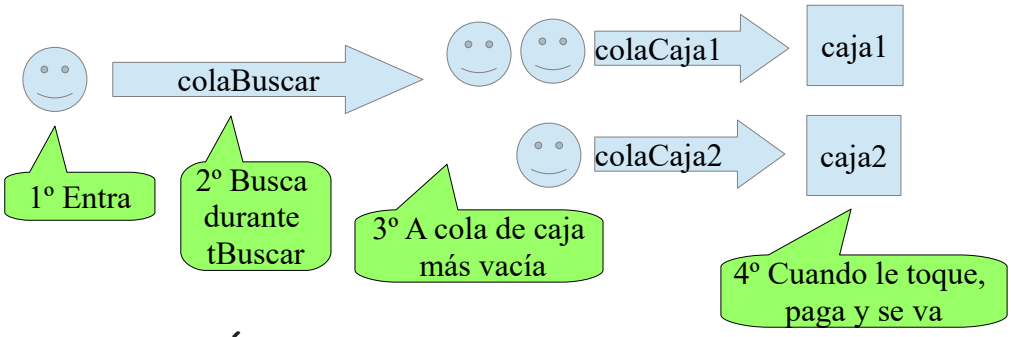
La empresa quiere saber para estas previsiones y configuración, algunos datos de funcionamiento (puedes ver los datos en la imagen de la ejecución de ejemplo), pero sobre todo les interesa saber el % de tiempo que van a pasar los clientes en la cola de las cajas y el tamaño medio de estas colas durante cada minuto de un día.

¿Cómo hacer un simulador que de forma aleatoria juegue a ver lo que sucede durante un día en este negocio? Bueno, nos ayudamos de colecciones de tipo cola y cola con prioridad. Tenemos una serie de variables que configuran el comportamiento de este mundo (variables de entrada), necesitamos un reloj que comience a valer 9 (cuando el negocio abre), se incremente con una precisión de 1 minuto (1/60 trozos de una hora) y acabe cuando llegue a 20 (hora de cierre). En cada minuto deben ocurrir cosas aleatorias, aunque siguiendo un patrón (entran clientes cada minuto siguiendo una distribución de Poisson), cada uno estará un tiempo aleatorio dando vueltas por el local (una cola con prioridad de la que salen cuando agotan ese tiempo), de ahí se van a la cola más vacía de las cajas y finalmente salen del local tras pagar.

Datos que necesita cada Cliente (una clase para usarla solo en el simulador):

```
public tEntra (tiempo en que entra en el local)
public tBuscar (minutos que estará dando vueltas por el local antes de ir a una caja)
public tQueda (tiempo que queda para acabar de buscar o de pagar)
public compra (nº de productos que compra),
public tCola (tiempo en el que entra en la cola de una caja)
public tCobrar (tiempo que se tarda en cobrarle)
public tSale (tiempo en el que sale de la caja y el local)
```

Comportamiento de los clientes:



ALGORITMO MÁS DETALLADO (Mira las notas)

```
LEER/DEFINIR variables de entrada (tasas de llegada clientes, …) y colas
reloj ← 9.0;
minuto = 1.0 / 60;          // Ritmo de avance del reloj de la simulación
MIENTRAS reloj <= 20 HACER
   cuantosEntran ← Nºclientes que entran en este minuto  // Nota 1 (sigue Poisson)
```

```
    PARA i = 1 HASTA cuantosEntran HACER
        Cliente c ← nuevo cliente(
                                tEntra ← reloj,
                                tBuscar ← (int)aleatorio ~ Normal(10, 5.8),    // Nota 2
                                tQueda ← tBuscar                               // Contador
                                compra ← (int)(tBuscar * aleatorio ~ Uniforme(0, 2)),
                                tCola ← 0,
                                tPagar ← (redondear a lo alto)(2 + compra / 4.0),
                                tSale ← 0
                            )
        Añadir cliente c a la cola con prioridad colaBuscar
    FINPARA i
    // TotalClientes += cuantosEntran
    Resta 1 al campo tQueda de todos los clientes de colaBuscar (los que menos les quede
    deben quedar antes en la cola)
    MIENTRAS haya clientes en colaBuscar con tQueda <= 0 HACER
        Quitar al cliente de colaBuscar
        tCola ← reloj
        tQueda ← tPagar       // Ahora cuenta si ha pagado
        Añadirlo a la cola más pequeña de las cajas
    FIN MIENTRAS
    PARA q = cada cola de las cajas HACER
        c ← el primer cliente de la cola
        c.tQueda ← c.tQueda - 1
        SI c.tQueda <= 0 ENTONCES
            Quitar a c de la cola   // El cliente ha pagado y sale
            c.sale ← reloj
            // Totalizar variables de salida de la simulación de los clientes:
            // tmBuscar, tmCaja, tmPagando, totalClientesAtentidos, ...
        FINSI
    FINPARA q
    // Actualiza tamaMedioCaja1 y tamaMedioCaja2
    reloj ← reloj + minuto
    SI reloj > 20 ENTONCES
        SI hay clientes en colaBuscar o en cajas ENTONCES
            vaciar y avisar  // No se les atiende
        FINSI
    FINSI
FINMIENTRAS
IMPRIMIR variables de salida de la simulación;
```

Nota1: Los clientes que entran durante cualquier minuto, es una variable aleatoria generada siguiendo una distribución de Poisson de media lambda (λ). Si de 9 a 10 se espera que entren 25 clientes, durante un minuto podrían entrar 25/60 clientes. Eso es el valor lambda de un minuto del día. Si las tasas las tienes definidas en el array tasaClientes[], puedes calcularla así:

λ = minuto * tasaClientes[(int) reloj - 9]

El algoritmo para transformar una variable aleatoria que sigue una distribución Uniforme(0,1) como los que devuelve un generador como Math.random(), en otra que siga la distribución de Poisson de media λ y una función en Java que la implementa:

```
ALGORITMO GETPOISSON(-λt)
DESCRIPCION: Calcula cuantos eventos ocurren durante el tiempo t, si
             siguen una distribución de Poisson de media λ
ENTRADAS: -λt   un valor double
SALIDA: un entero indicando el nº de ocurrencias
PASO 1: Sean k = 1, p = 1
PASO 2: Se genera  u ~ U(0,1) y se hace k = k u;
PASO 3: SI k ≤ exp(-λt) ENTONCES DEVUELVE x - 1.
PASO 4: x = x + 1
PASO 5: IR AL PASO 2.
```

```
public static int getPoisson(double lambda) {
    double L = Math.exp( -lambda );
    double p = 1.0;
    int k = 0;
    do {
        k++;
        p = Math.random();
    } while ( p > L );
    return k-1;
}
```

Nota 2: para generar números aleatorios que sigan una distribución Normal(media, desviación) a partir de dos que sigan una distribución Uniforme(0,1) te propongo que uses este método:

Usamos r1 ~ U(0,1) y r2 ~ U(0,1) (La letra ~ debes interpretarla como que siguen …)

$$z_1 = \sqrt{-2\ln(rI)}\sin(2\pi r_2)$$

$$n_1 = z_1\sigma + \mu$$

La variable n1, generada tal como indican las anteriores expresiones, sigue una distribución normal de media μ y desviación σ.

Ejemplo de ejecución:

```
Se cierran las cajas. Pendientes en caja1 7 y en caja2: 6 en local: 10

RESUMEN DE LA SIMULACIÓN:
 - Local abierto de 9:00 a 20:0
 - Entran 737 clientes
 - Compran 9,79 productos de media
 - Pasa 10,20 minutos buscando productos
 - Pasa 4,58 minutos en la cola
 - Pasa 4,12 minutos pagando
 - Tiempo por cliente: COMPRANDO: 53,97% COLA: 24,22% PAGANDO: 21,80%
 - Tamaño cola caja 1 3,32 clientes/min
 - Tamaño cola caja 2 2,92 clientes/min
```

EJERCICIO 5. IMPLEMENTAR ITERADORES Y COMPARADORES.

a) (0.3 PUNTOS) Implementa un conjunto de tipo diccionario (ordena objetos y no permite repetidos) usando un árbol binario de búsqueda genérico. No hace falta que sea balanceado. Puedes usar el código que hay en el tema.

b) (0.4 PUNTOS) Define la clase NodoS (tiene un String y un int) y que implemente comparable<NodoS>. El comparador por defecto usa el String (a modo de clave). Usa el diccionario para almacenar elementos NodoS y cuenta las palabras del fichero del ejercicio 1, es decir lo usas como Diccionario<NodoS>.

c) (0.3 PUNTOS) Implementa un iterador para los elementos del Diccionario y recorre todos los elementos en orden (de los String) imprimiendo las 10 primeras palabras que encuentres que tengan más de 5 letras e imprime también su frecuencia de aparición.

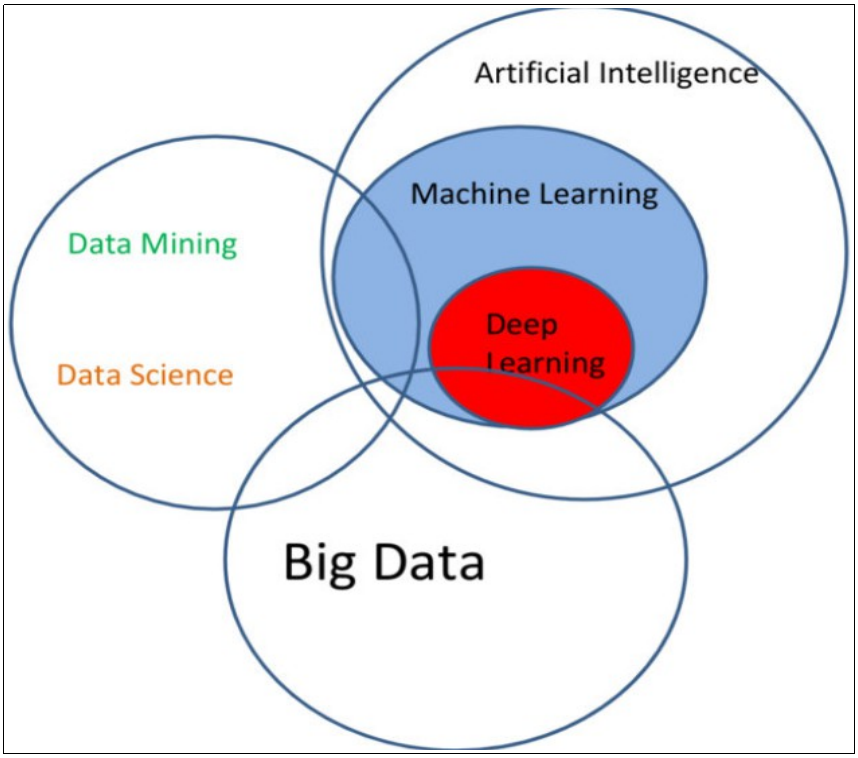
Pasa código y una captura de pantalla de la ejecución.

EJERCICIOS JEDI

Como siempre, solo debéis entregar el que os toque, aunque es recomendable intentar resolver todos para practicar.

EJERCICIO 6. SISTEMA DE RECOMENDACIÓN.

Actualmente hay varias tecnologías que están de moda y que se dedican a intentar extraer datos de alto nivel a partir de la información que generan las organizaciones (mira la figura). ***El DATAMINING (minería de datos)*** es el término de informática que define unas tecnologías que se especializan en recolectar datos de diferentes fuentes y formatos (BD relacionales, BD NoSQL, ficheros XML, ficheros raw, JSON, servicios en red, ficheros, programas, sensores, imágenes, sonidos, etc.), pre-procesarlos para adaptarlos/organizarlos a como se necesitan para poder analizarlos empleando diferentes técnicas (estadística, OLAP, machine-learning, deep-learning)) y así obtener nueva información útil para tomar decisiones sobre algún aspecto interesante para la empresa.



Se aprovechan de la información que ya no sirve a la empresa para realizar su actividad cotidiana: a quien le vende algo, qué le vende, a qué precio, si ya ha cobrado o está pendiente, donde se carga el importe, a donde se envía, en qué formato (papel, ebook), etc.). Esa información ya no es útil para la actividad cotidiana de la empresa, pero ahora se utiliza para estudiarla y sacar nueva información que está oculta en esos datos. Igual que la minería tradicional extrae minerales interesantes a partir de las rocas que excava, mueve, procesa y trata de mil maneras, etc. datamining hace lo mismo, pero con datos.

Vamos a ver un ejemplo de aplicación de estas tecnologías. Imagina una empresa de venta de libros on-line que quiere implementar un sistema de recomendación, porque claro, la empresa tiene unos 2 millones de libros y añade cada año unos 15.000 nuevos. Así que entre tanta oferta, para un cliente es muy difícil elegir que es lo que quiere leer a continuación. Si no hiciera un sistema de recomendación, quizás muchos clientes irán perdidos entre tanta oferta y acaban por no encontrar algo que posiblemente les interese y la empresa perdería negocio.

Vas a implementar un sistema de recomendación sencillo denominado ***Sistema de Filtrado Colaborativo por Similitud de Usuarios***. La verdad es que este tipo de sistema tiene muchos inconvenientes comparado con otros más avanzados:

- ***Es un sistema basado en memoria***: debe chuparse todos los datos cada vez que hace una recomendación. Hay otros (basados en modelos) que no necesitan hacerlo → más rápidos → más escalables.
- ***Son sensibles a las manías de los usuarios***: algunos usuarios son generosos y no puntúan nunca bajo, otros son excesivamente tacaños, otros mienten sobre sus preferencias, otros directamente pasan de puntuar. Los sistemas más sofisticados recopilan lo que los usuarios hacen en vez de trabajar con lo que dicen y otros aún más sofisticados se basan en las características de cada producto (filtrado basado en ítems).
- ***Son sensibles a datos dispersos***: como hay tantos artículos (libros en nuestro ejemplo), la probabilidad de que 2 usuarios hayan leído y puntuado libros en común es muy baja → Si hay una oferta de un millón de libros, y de media cada cliente compra y puntúa durante toda su vida 30 libros (la probabilidad de que un usuario haya puntuado un libro es 30/1000000 = 0.00003).
- ***Respuestas sensibles a las dimensiones que haya***: pueden encontrar falsos parecidos debido a lo anterior, debido la métrica de distancias que use y otras causas en las que no vamos a entrar.

Bien, pese a estos inconvenientes, la ventaja que tienen este tipo de sistemas de recomendación es que son "sencillos" de entender, así que como esto es solamente una excusa para programar un poco, veamos su algoritmo general:

-- Sistema de Filtrado Colaborativo por Similitud de Usuarios
PASO 1: Leer un cliente <x> al que hacer recomendaciones.
PASO 2: Buscar el/los k clientes más parecidos (los llamamos <cercanos>) a los gustos expresados por <x>.
PASO 3: Recomendarle los productos que mejor hayan puntuado los clientes <cercanos> y que no haya puntuado el cliente <x>.

A simple vista, la mayor dificultad la vamos a tener en el PASO2. **¿Cómo saber cuál o cuales son los usuarios más parecidos en gustos al usuario <x>?** Bien, nos ponemos en el ejemplo de la venta de libros on-line. Imagina que tienen 4 usuarios (Amy, Bill, Jim y x) y 2 libros que son "Snow Crash" y "Dragon Tattoo" al que nosotros conocemos por **"Los hombres que no amaban a las mujeres")**. Tenemos las puntuaciones de 0 a 5 que le han dado estos usuarios a

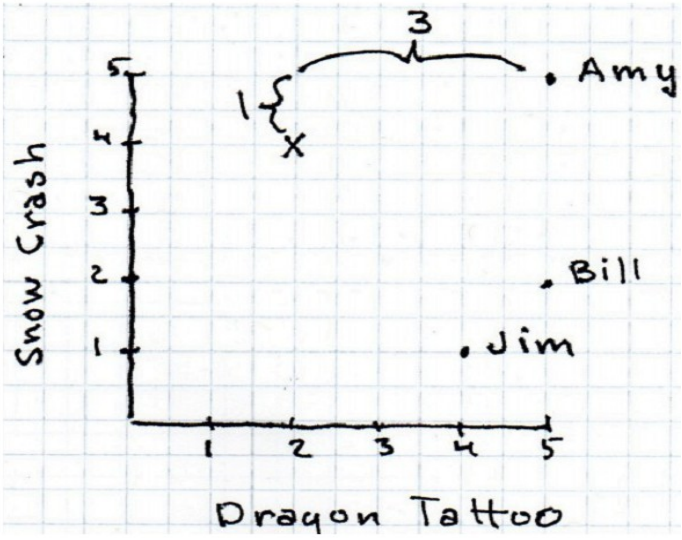
esos libros resumidas en esta tabla (0 es que no hay valoración, 1 es la peor valoración y 5 la mejor):

Clientes	Snow crash	Dragon Tattoo
Amy	5	5
Bill	2	5
Jim	1	4
x	4	2

¿Cuál es el cliente más parecido a x? Para cada usuario que tenga libros puntuados en común con él (en el ejemplo todos tienen estos 2 libros puntuados), vamos a calcular la distancia entre ellos en 2 dimensiones (cada libro puntuado en común se considera una dimensión y llamamos a este valor n).

Usamos una variación de la **métrica manhattan** que es: si usuario1 puntúa con u_1 el primer libro y con u_2 al segundo libro y …. y con u_n al n-ésimo libro y el usuario2 puntúa con v_1 al primer libro y con v_2 al segundo, … y con v_n al n-ésimo, la distancia manhattan entre los usuarios será:

$$\text{distancia manhattan(usuario1,usuario2)} = \frac{(\,|u1 - v1| + |u2 - v2| + \ldots + |un - vn|\,)}{n}$$



Nota: |k| es el valor absoluto del número k. Función abs(k).

Así que:

```
manhattam(x, Amy) = ( |5-4| + |5-2| )/2 = ( |-1| + |3| )/2 = (1 + 3)/2 = 2
manhattam(x, Bill)= ( |2-4| + |5-2| )/2 = ( |-2| + |3| )/2 = (2 + 3)/2 = 2.5
manhattam(x, Jim) = ( |1-4| + |4-2| )/2 = ( |-3| + |2| )/2 = (3 + 2)/2 = 2.5
```

El usuario más parecido a <x> será el más cercano a sus gustos, es decir, el que menor distancia tenga, que en este caso es Amy con una distancia de 2, mientras que Bill y Jim están a una distancia de 2.5 (son menos parecidos).

Bien, aunque en la realidad sería un poco más complejo (no es la única ni la mejor métrica, el valor será más o menos fiable en función de cuantas puntuaciones se utilicen para calcularla, etc.) esas otras cuestiones no las vamos a tener en cuenta.

1ºDAM PROGRAMACIÓN
PRÁCTICA UNIDAD 8: GENÉRICOS Y COLECCIONES

Crea el programa U08E06.java que implemente un sistema de recomendación de chistes de una empresa que tiene a humoristas de clientes. Para ello:

- Define la clase usuario: Id + nombre
- Define la clase Chiste: Id + texto
- Define la clase Puntos: Chiste + usuario + puntos (1-5)
- Crea un programa que permita Cargar colecciones de usuarios, chistes y puntuaciones desde los ficheros: usuarios.csv y chistes.csv, puntos.csv (tiene los Id no los chistes ni los usuarios)
- Escoge 3 chistes al azar y haz que el usuario del programa los puntúe de 1 a 5.
- Aplica el algoritmo de recomendación para buscar los usuarios más parecidos a él (si los hay) y los imprimes.
- Recomienda los chistes puntuados por el usuario más parecido que no haya puntuado el usuario actual e indica un valor de recomendación como: puntos dados por el usuario cercano * parecido entre ellos.

Muestra código y captura de la ejecución. Crea y usa estos ficheros:

Usuarios.csv	Chistes.csv	Puntos.csv
1;Eldelbar 2;Eldelgas 3;Epi 4;Blas 5;Calamardo	1;¿En que se parecen Cristiano y Pinocho?\nEn que los dos son de Madeira 2;Le dice un león a otro: no te acerques a las cebras que están rayadas. 3;¿Cuál es el colmo de un fotógrafo?\nQue su hijo sea negativo. 4;¿Qué hace Enrique Iglesias en un trigal?\nEstar con trigo, vivir con trigo... 5;Se levanta el telón y se ve a Paca electrocutada ¿Cómo se llama la película?\nEl amperio contraPaca. 6;-Te detesto.\n-K de kilo.\n-¿Qué dices?\n-No sé... has empezado tú. 7;-No tienes ni idea de lo que soporto.\n-Una ciudad de Portugal. 8;Todo eran risas, hasta que nos dimos cuenta que el tartamudo quería JAMÓN. 9;Estoy leyendo un libro nuevo que se llama \"La honestidad y otros valores\".\n-Que bien, ¿dónde lo has comprado?\n-Lo robé de la librería de la esquina. 10;-Johnson de homicidios,¿quién es?\n-Agente Aguirre.\n-¿Qué ha pasado?\n-Asesinato de un varón de 38 años. Su madre le ha dado 6 puñaladas por pisar lo fregao.\n-Han detenido a la madre?\n-No, todavía está mojado. 11;-¿Tienes wi-fi?\n-Sí\n-¿Y cuál es la clave?\n-Tener dinero, llamar a una operadora y contratarlo, no tiene más... 12;Se abre el telón y se ve un tetrabrik de Don Simón. ¿Cómo se llama la película?\n-Es tinto básico. 13;En Bilbao, al salir de un examen de Análisis Matemático:\n-Oye Patxi, ¿qué te ha dado el segundo límite, pues?\n-Infinito.\n-	1;2;2 1;3;5 1;4;4 1;7;3 1;8;1 2;1;2 2;3;1 2;5;4 2;7;5 2;9;5 3;11;3 3;13;2 3;15;1 3;17;4 3;2;3 4;4;4 4;6;1 4;8;5 4;10;4 4;12;2 5;14;1 5;16;2 5;1;4 5;3;2 5;5;5

	¿Sólo? 14;-Juan, ¿Qué es peor, la ignorancia o la indiferencia?\n-Ni lo se, ni me importa. 15;-¿Qué tal tu viaje por Italia?\n-Muy bien, estuve en Roma, en Milán, en Temeo.\n-¿Temeo? ¿Nunca había oído hablar de esa ciudad?\n-Pues es grandisima.\n-¿Si?\n-Si, y muy famosa.\n-¿Estás seguro de que se llama Temeo?\n-Ah no! perdona, era Torino. 16;-Buenas ¿Tiene la película llamada \"Tu culo está estacionado\"?\n-¿Sabe el título en inglés?\n-Yourassispark. 17;-¿Y vosotros los Jedis, la espada láser dónde la compráis?\n-Aparque en el arcén, ya no hace falta ni que sople...	

EJERCICIO 7. USAR LIST<T>.

Supongamos que nos encargan realizar un software que sea capaz de detectar preguntas copiadas en trabajos. Dados dos textos (que son la contestación a una pregunta), tendremos que pensar como medir lo parecidos o diferentes que son. Lejos de ser algo novedoso, estas técnicas se utilizan habitualmente en motores de búsqueda (google y muchos más), y no solo para buscar páginas web, también en bases de datos, etc..

Normalmente se utiliza el término distancia o similitud. La distancia es una medida numérica que nos indica lo diferentes que son: cuanto mayor sea la distancia que guardan dos textos, más diferentes son. Esta medida puede normalizarse para que vaya de 0 a 1 (no siempre), donde 0 significa que son exactamente iguales y 1 significa que no tienen nada en común. La similitud es lo contrario, a mayor similitud más parecidos y a menor similitud más diferentes: similitud = 1.0 – distancia (si la distancia está normalizada).

Para experimentar, vamos a coger una pregunta de un trabajo de Economía, relacionada con los organismos de defensa del consumidor. La pregunta es esta: **¿Qué empresas están obligadas a adherirse a un sistema arbitral de consumo?** Y tenemos 3 respuestas:

> Texto1 = "Adherirse al sistema arbitral es voluntario e incluso pueden solicitar su adhesión dejando al margen ciertos conflictos. Aunque sin estar inscrita, podría aceptar este mecanismo en algún caso.

Otra respuesta es una copia a la que han cambiado el principio de la primera frase:

> Texto2 = "Unirse es voluntario e incluso pueden solicitar su adhesión dejando al margen ciertos conflictos. Aunque sin estar inscrita podría aceptar este mecanismo en algún caso."

Y otra respuesta es una nueva copia, pero ahora han cambiado el orden, primero escriben la segunda frase y luego la primera.

> Texto3 = "Aunque sin estar inscrita podría aceptar este mecanismo en algún caso pero adherirse al sistema arbitral es voluntario e incluso pueden solicitar su adhesión

dejando al margen ciertos conflictos. "

Hay medidas (de distancia/similitud) que se fijan en el léxico. Si tomamos como elemento unidad la palabra, miden la coincidencia de estos elementos, o cuantas operaciones (insertar, eliminar, modificar) son necesarias para transformar un texto en el otro. Estos no detectarían copias que consistan en cambios de posición de trozos. Otras distancias se centran en la semántica. Hay dos tipos: basadas en corpus y en conocimiento. El corpus consiste en un conjunto de textos de los que se extrae información estadística del lenguaje que se utiliza. Las semánticas basadas en conocimiento utilizan bases de datos de elementos del lenguaje y son los más efectivos y complejos de implementar.

Nosotros, usaremos este pretexto para practicar con arrays y colecciones de Java y vamos a calcular dos distancias: D1 es una distancia léxica denominada el algoritmo de Levenshtein y otra D2 que es semántica basada en corpus, denominada el coeficiente de Jaccard. Pero para simplificar el ejercicio, no utilizaremos corpus.

Crea un programa que defina las 3 respuestas anteriores e implemente estos métodos:

a) (0.4 PUNTOS) Calcular D1: Una versión de la distancia de Levenshtein (normalizada a 0-1). Utiliza solamente arrays y métodos de la clase **Arrays** .

```
DOUBLE coeficienteLevenshtein(String texto1, String Texto2)
    t1 ← Array de palabras de texto1 pasadas a minúsculas sin signos de puntuación
    t2 ← Array de palabras de texto2 pasadas a minúsculas sin signos de puntuación
    declara d[t1.longitud+1, t2.longitud+1]
    declara int i, j, coste
    para i desde 0 hasta t1.longitud haz d[i, 0] ← i; finpara
    para j desde 0 hasta t2.longitud haz d[0, j] ← j; finpara
    para i desde 1 hasta t1.longitud haz
        para j desde 1 hasta t2.longitud haz
            si t1[i-1] = t2[j-1] entonces coste ← 0 sino coste ← 1; finsi;
            d[i,j] ← minimo(
                        d[i-1, j] + 1,      // borrado
                        d[i, j-1] + 1,      // inserción
                        d[i-1, j-1] + coste   // sustitución
                    )
    finpara
finpara
devuelve d[t1.longitud, t2.longitud]/maximo(1.0, ABS(t1.longitud - t2.longitud))
```

b) (0.4 PUNTOS) Calcular D2: El coeficiente de Jaccard es más sencilla de calcular si usas colecciones de Java. Por tanto ahora no utilices arrays (más que de forma auxiliar), mejor utiliza algunas listas por ejemplo, e investiga sus métodos para calcular lo que necesites.

```
double coeficienteJaccard(String texto1, String texto2) { ...
       PASO1: Calcular:
              a = Nº de palabras comunes en los dos textos.
              b = Nº de palabras que están en texto1 y no en texto2.
              c = Nº de palabras que están en texto2 y no en texto1.
```

PASO3: calcular y devolver $\frac{a}{a + b + c}$

c) (0.2 PUNTOS) El programa debe imprimir:

```
Texto1 y Texto2:
       Similitud Lexica (Levenshtein):  1 - D1
       Similitud Semantica (Jaccard):   1 – D2
       Media: ...
Texto1 y Texto3: (lo mismo) …
Texto2 y Texto3: (lo mismo) ...
```

EJERCICIO 8. ITERADORES Y COMPARADORES.

a) (0.2 PUNTOS) Define una clase llamada **Municipio** que contenga el nombre de una población y sus habitantes. (Crea constructores, getters, setters y sobreescribe toString()).

b) (0.2 PUNTOS) Queremos manipular estos datos mediante una colección Lista. Crea una lista y añade estos y calcula la media de la población usando un iterador.

> **{"Ademuz", 1149} {"Ador", 1374} {"Adzaneta de Albaida", 1307}, {"Agullent", 2402} {"Alaquàs", 29838} {"Albaida", 5987} {"Albal", 16029} ...**

c) (0.3 PUNTOS) Modifica la clase Municipio para que implemente la interfaz Comparable. Haz que un municipio sea menor que otro si su población es menor, mayor si lo es su población o igual solo si su población es igual y también lo es su nombre. Muestra los municipios de la colección ordenados (Si lo pides a la colección, usará compareTo() de la clase que almacene) recorriéndola con un bucle while y un iterator o un bucle for-each.

d) (0.2 PUNTOS) Ahora imagina que queremos los elementos ordenados por un criterio distinto al que tiene la clase, por ejemplo por orden alfabético. Crea un objeto de tipo Comparator y pide a la clase Collection que te ordene la lista por este nuevo criterio, y la imprimes mediante un bucle for-each.

e) (0.1 PUNTOS). ¿Cómo definirías el comparador del apartado b) mediante una función lambda?

EJERCICIO 9. USAR PILAS.

Una pila (stack) es una lista LIFO, una lista normal en la que solamente utilizas dos operaciones push() para añadir un elemento y pop() para sacar un elemento. Como ejemplo de utilización de pilas vamos a hacer un programa que sea capaz de leer un string que contenga una serie de operaciones parentizadas y sea capaz de realizar el cálculo que indique la expresión.

2 * 3 + 4 → No se puede salvo que crees un compilador

((2 * 3) + 4) → podemos hacerlo sin compilador (cada operación un paréntesis)

E. W. Dijkstra desarrolló un algoritmo en 1960 que lo hace usando dos pilas, una pila de operandos (valores) y otra pila de operaciones. Una expresión contiene paréntesis, operandos (números) y operaciones. Recorriendo la expresión de izquierda a derecha y extrayendo cada uno de estos elementos uno a uno se hace lo siguiente:

- si encuentras un número: push(numero) en la pila de operandos.
- Si encuentras un operador: push(operador) en la pila de operaciones.
- Si encuentras un paréntesis izquierdo "(": lo ignoras.
- Si encuentras un paréntesis derecho: operacion= operaciones.pop(), sacar de la pila de operandos los valores que necesite la operación, realizar el cálculo y el resultado se vuelve a añadir a la pila de operandos.

Cuando se acaben los elementos de la expresión, si está escrita de forma correcta, en la pila de operandos quedará un valor que es el resultado de calcular la expresión.

Nota: si no separas con espacios cada elemento de la expresión, necesitarás un analizador léxico que reconozca los elementos (llamados tokens) admitidos en el lenguaje (números, operaciones y paréntesis en este caso).

Nota: si no encierras entre paréntesis cada expresión, necesitarías un traductor (intérprete o compilador) que coge los tokens (elementos) de la expresión, construye un árbol sintáctico de la expresión (lo controla la sintaxis del lenguaje) y lo transforma en operaciones que pueda implementar, por ejemplo en pilas de operaciones y valores.

Crea el programa en Java que lea la expresión de la entrada estándar (si la hay) o pida un String (si no se indica). Ejemplos:

```
C:\> java U8E3 ( 1 + ( ( 2 + 3 ) * ( 4 * 5 ) ) )
101.0

C:\> java U8E3
Teclee la expresión: ( ( 1 + raiz2( 5.0 ) ) / 2.0 )
1.618033988749895
```

EJERCICIO 10. IMPLEMENTA LA TABLA HASH.

A partir del apartado 8.5.4 de la unidad, define una interfaz genérica de una tabla hash y una clase también genérica que la implemente. Haz un programa de prueba.

EJERCICIOS OPCIONALES

EJERCICIO 11 (OPCIONAL +2). ITERADORES EN ÁRBOLES.

Mira el apartado de árboles binarios en el tema y:

a) (0.5 Puntos) Define una interfaz y una clase genéricas de árboles binarios.

b) (0.5 Puntos). Define un iterador que recorra el árbol en inorden.

c) (0.5 Puntos). Define un iterador que recorra el árbol en preorden.

d) (0.5 Puntos) Define un iterador que recorra el árbol en postorden.

Todos los iteradores deben tener las operaciones: hayAnterior(), haySiguiente(), anterior(), siguiente().

Crea uno y prueba a utilizar la sentencia for-each para recorrerlo.

OPCIONAL (+ 5 PUNTOS). CÓDIGOS DE HUFFMAN.

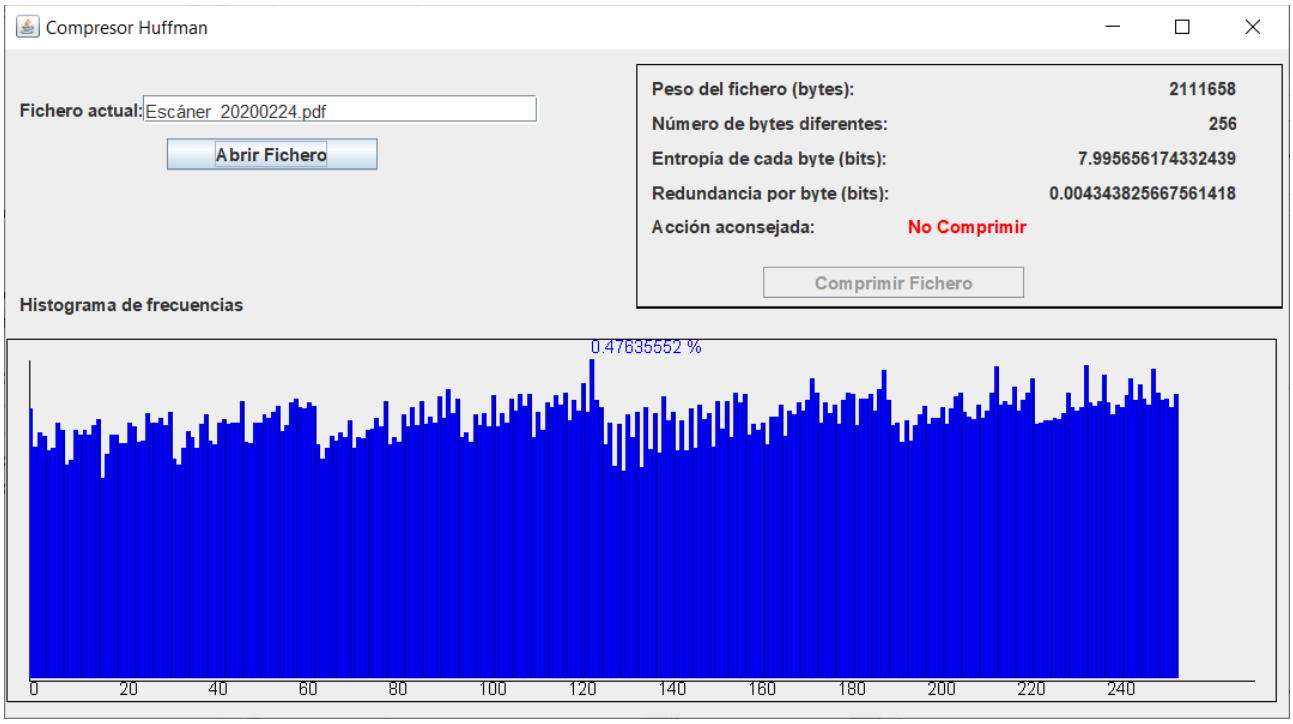
Cualquier fichero (sea de texto de binario) a fin de cuentas no es más que una serie de bytes formados por bits (0 y 1). A veces es posible modificar la forma en que se codifican estos bytes para conseguir que esa información pese menos (reducir la cantidad de bytes almacenados) este proceso se conoce como comprimir la información y es lo que hacen formatos como zip, .arj y otros.

Si pensamos en un fichero de texto con letras codificadas en ASCII, cada letra tiene 8 bits de peso asignados. Cada letra pesa un byte y cada byte tiene valores de 0 a 255. Una manera de comprimir consiste en cambiar estos códigos de tamaño fijo por otros códigos (bits) de tamaño variable, de forma que tengan menos bits, los valores (bytes) que más aparezcan en el fichero. Una forma sería usar un prefijo de tamaño diferente:

Byte que más aparece	Bits usados para escribirlo
B1	1
B2	01
B3	001
B4	0001

Pero una mejora, para diferentes bytes tener prefijos de igual longitud, se consigue utilizando la codificación de Huffman es una forma de aplicar esto.

PASO 1 (1 PUNTO): Debes hacer un programa GUI en Java que presente una interfaz en WindowBuilder similar a esta:



Cuando elijas un fichero, debe abrirlo como binario, recorrerlo y crear un histograma de los bytes que contiene, que no es más que un array de enteros de 256 valores donde en cada posición, cuentas las veces que aparece ese byte en el fichero (la posición 0 contiene cuantas veces aparece el byte 0, la posición 1 cuenta cuantas veces aparece el byte 1 en el fichero, …) así hasta la posición 255.

Luego hará un gráfico de barras que visualice el histograma. Para que se aprecie la distribución de bytes, se calcula la máxima frecuencia de aparición (el mayor valor del array de la tabla de frecuencias) y se grafican las barras de cada frecuencia de forma que la mayor coincida con la altura de la zona del gráfico menos 20 píxels. Al lado de la barra de la mayor frecuencia, escribimos la frecuencia (en la figura 0.47…%).

El programa calcula la entropía de la información que hay en el fichero, que mide de media, cuantos bits en cada byte tienen realmente información, y cuantos bits de media son redundantes (sobran) y por tanto podrían ser prescindibles. La entropía máxima del fichero podría llegar a 8. Cuanto más cerca de 8 esté, menor será la redundancia (8 – entropía) y menos beneficioso es comprimirlo porque no conseguiremos bajar su peso de forma significativa (ya está comprimido). El programa debe desaconsejar comprimirlo mostrando un mensaje en rojo.

> **(0.5 PUNTOS) ACTIVIDAD:** Coge un fichero de texto sin comprimir (pero que no sean 4 letras, por ejemplo, copia el texto de esta práctica y lo pasas al bloc de notas y lo guardas como texto1.txt). Luego lo comprimes en texto2.zip. Ahora haz un programa que genere valores aleatorios entre 0 y 255 y los guardas de forma binaria en el fichero texto3.bin. Abre los tres ficheros con tu programa y muestra capturas de pantalla y es aconsejable o no comprimirlos. ¿Cuál tiene menor redundancia? ¿Un fichero aleatorio es similar a uno comprimido?

PASO 2 (1 PUNTO): DEFINIR UN ARBOL HUFFMAN.

Es un enlace a un nodo interno o Hoja que en realidad define un árbol binario donde los nodos hoja contienen un elemento E y un peso y los nodos intermedios solo contienen un peso que es la suma de sus hijos.

Construye una clase genérica para un nodo del árbol de Huffman que se llame **NodoBase<E>** que obligue a implementar las operaciones:

```
public boolean esHoja();      // Devuelve true si es un nodo hoja o false si no lo es
public int getPeso();         // Devuelve el peso del nodo (la frecuencia)
```

Ahora haz dos nuevas clases que deriven de la abstracta/interfaz llamadas NodoHoja y NodoInterno para diferenciar las hojas de los nodos intermedios que tienen diferente contenido.

NodoHoja	NodoInterno

Private E e; // elemento
private int peso; // peso
constructor(e, p)
getElemento() // devuelve el elemento e
getPeso() // devuelve el peso
esHoja() // devuelve true
	private int peso; // peso
	private NodoBase<E> hI; // hijo izq
	private NodoBase<E> hD; // hijo dere.
	constructor(NodoIzq, NodoDer, peso)
	getPeso() // devuelve el peso
	esHoja() // devuelve false
	getIzq() y getDer() // devuelve hijos

Un árbol Huffman será una referencia al nodo que hace de raíz de un árbol y sus elementos deben ser comparables. Lógicamente lo haremos genérico. Tendrá estos elementos:

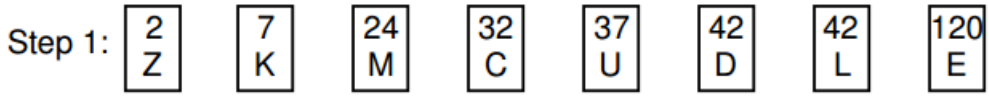
```
class ArbolHuffMan<E> implements Comparable<ArbolHuffman<E>> {
        private NodoBase<E> raiz;

        // Constructores
        ArbolHuffman(E elemento, int peso) {
            raiz = nuevo nodoHoja<E>(elemento, peso);
        }

        ArbolHuffman( NodoBase<E> i, NodoBase<E> d, int peso) {
            raiz = nuevo NodoInterno<E>(i, d, peso);
        }

        public NodoBase<E> getRaiz() { ….

        public int getPeso() {  // devuelve el peso de la raíz....

        public int compareTo( ArbolHuffman<E> otro) {
                // devuelve: -1 si this.raiz.peso < otro.raiz.peso
                //           +1 si      "         >      "
                //            0 si son iguales
        }
}
```

PASO 3 (1 PUNTO): PROCESO DE CREAR ÁRBOLES DE HUFFMAN.

Imagina que tenemos la siguiente tabla de frecuencias donde solo aparecen los bytes con frecuencias mayores de cero de un fichero (los bytes que contiene un fichero no tienen

porqué ser todos) y ten en cuenta que en la figura se ven letras en vez de el valor numérico de cada byte. La tabla de la figura indica que el carácter 'C' (o su valor numérico equivalente) aparece 32 veces, la 'D' aparece 42 veces, etc.

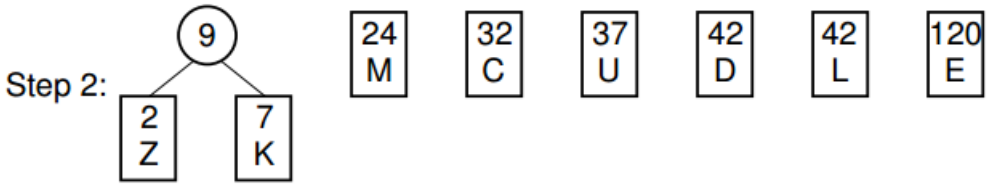
Letter	C	D	E	K	L	M	U	Z
Frequency	32	42	120	7	42	24	37	2



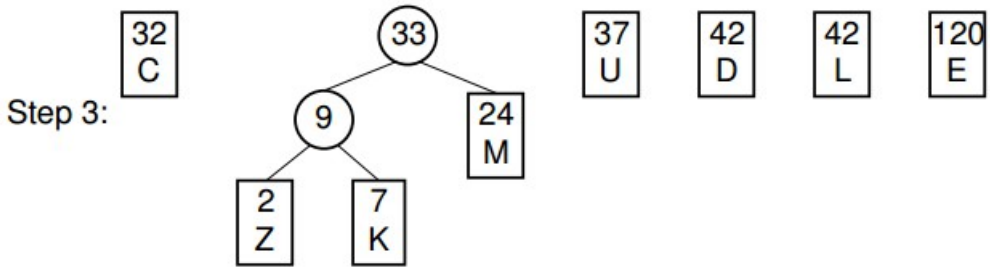
Crearemos un Árbol Huffman para cada entrada en la tabla que tenga un peso donde se almacena la frecuencia de los bytes con frecuencias mayores de cero. El nodo tiene el valor del byte y el peso (cantidad de veces que aparece en el fichero) y lo **añadimos a una cola con prioridad que almacena árboles binarios (nodos con: byte + freq + hi + hd + padre) ordenada por la frecuencia**. De esta forma, la cabeza de la cola (por donde desencolamos elementos) queda en los valores bajos: ← 2, 7, 24...

```
Mientras haya más de un elemento en la cola hacemos:
        desencolar en hijo izquierdo (variable iz)
        desencolar en hijo derecho (variable de)
        nuevo ← nuevo árbol binario(frec← iz.frec + de.frec, hijoI ← iz, hijoD ← de);
        Encolar nuevo
FinMientras
```

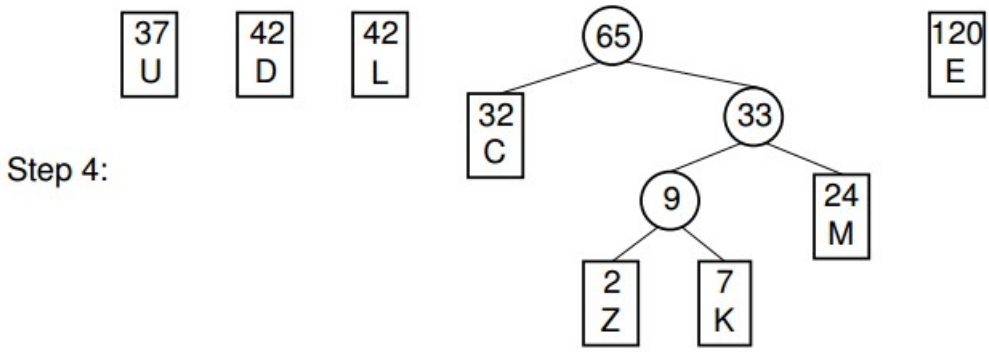
Este trozo de código irá construyendo un árbol binario (de Huffman). Por ejemplo, la primera vez combina el byte 'Z' y el byte 'K' en un árbol con peso 9 (7 + 2) que al final de la iteración se vuelve a encolar y debe quedar en la cabeza de la cola (porque 9 es el peso más bajo):



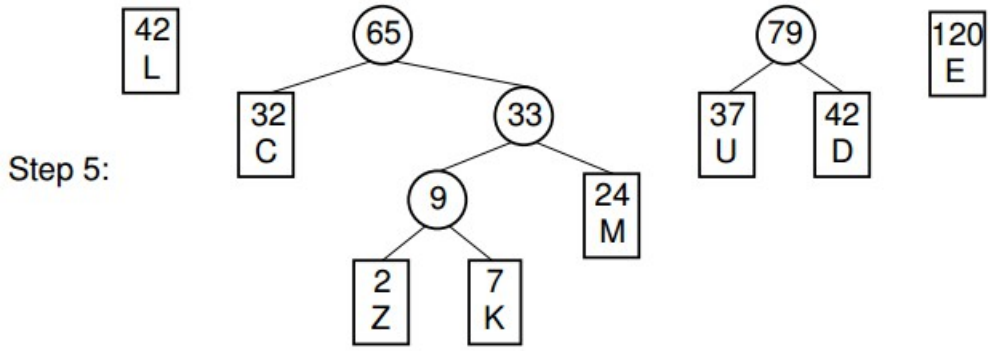
Como hay más de 1 elemento, vuelve a repetirse y en la segunda iteración la cola queda así:



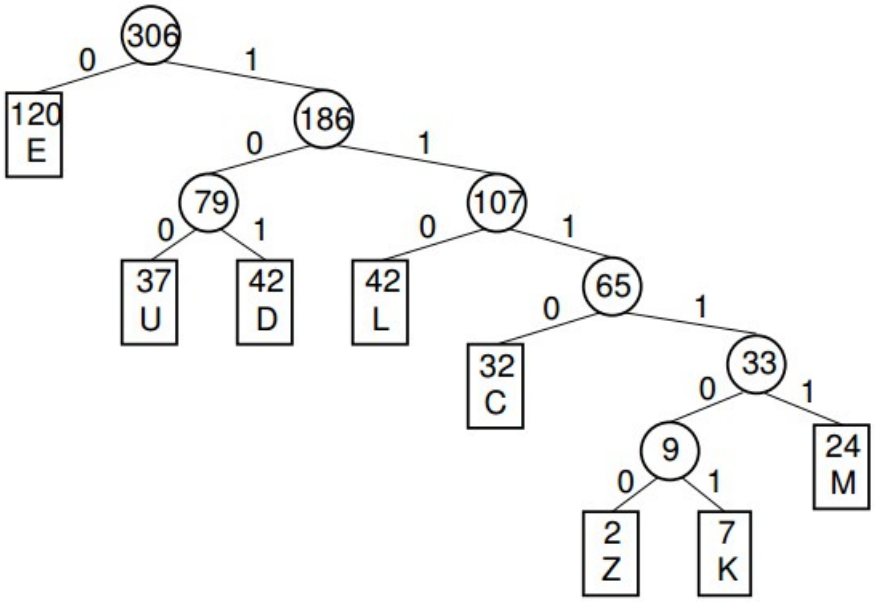
En la tercera iteración del mientras:



En la cuarta iteración:



Y al final, conseguimos tener la cola con prioridad con un solo elemento, con el árbol de Huffman completado. En este caso se nos queda así:



Este proceso de construcción es un ejemplo de algoritmo de **greedy**. En cada paso, el algoritmo toma la decisión de mezclar los dos subárboles de menor peso, lo que da el resultado más óptimo. Si para cada nodo hoja, sacamos el recorrido hasta llegar desde la raíz a él, de forma que si vamos a la izquierda ponemos un 0 y si vamos a la derecha un 1, podemos generar los prefijos binarios por los que vamos a sustituir cada byte o letra o palabra:

Letter	Freq	Code	Bits
C	32	1110	4
D	42	101	3
E	120	0	1
K	7	111101	6
L	42	110	3
M	24	11111	5
U	37	100	3
Z	2	111100	6

PASO 4 (1 PUNTO). GENERAR NUEVO FICHERO CON INFORMACIÓN COMPRIMIDA.

Una vez que tenemos el árbol de Huffman construido, es fácil asignar los códigos a los bytes individuales. Nos hacemos un array de Strings de esta forma:

- **El valor de cada byte:** si el array es de 256 elementos (los String que almacenan estarán a null por defecto) podemos asociar el índice del array a cada byte posible (el byte 23 estará en la posición 23 del array, el 52 en la posición 52, etc.) y así accedemos rápidamente a su código y como los que están a null no van a utilizarse nunca, no hay problema.
- **Código binario de cada byte:** Un String con la cadena binaria (caracteres 0 y 1) que les corresponde en la nueva codificación Huffman.

Para calcular los códigos, comenzando en la raíz del árbol, asignamos '0' o '1' a cada enlace del

árbol ('0' si nos vamos a la izquierda y '1' si nos vamos a la derecha). El código del byte que hay en cada nodo hoja será la ruta desde la raíz hasta el nodo hoja. El código para el byte 'E' será '0' porque la ruta hasta su nodo desde la raíz implica recorrer un enlace a la izquierda. El código para el byte 'K' es "111101" por el mismo motivo. Por tanto para obtener la tabla de códigos tendrás que recorrer el árbol y cuando estés en un nodo hoja, anotar el recorrido que llevas en la tabla de códigos del byte.

Para generar el fichero comprimido, y poder luego descomprimirlo y recuperar la información original debemos guardar no solo el mensaje comprimido sino la información que nos permita deshacer el proceso. Le vamos a dar el siguiente formato:

HUF	EXT	N	B	Tabla de frecuencias	Mensaje comprimido

Donde:

- **HUF (siempre 3 bytes)**: son los caracteres 'H' 'U' y 'F' una especie de marca que servirá a nuestro programa para reconocer que es un fichero generado por él. Sus ficheros acabarán con la extensión .huf pero podrían cambiarle la extensión a otro fichero e intentar reventarlo, para evitarlo, ponemos esta marca como una medida de protección.
- **EXT (siempre 4 bytes)**: Guardaremos 4 caracteres con la extensión que tenga el fichero original para que al descomprimir sepa que extensión ponerle (un "txt ", "png ", "pdf ", "docx", "jpeg", "html", …). Si la extensión original tiene solo 3 letras, añadimos un espacio en blanco al final para rellenar hasta 4.
- **N (siempre 8 bytes):** un long que contiene cuantos bytes pesa el mensaje original sin comprimir, para verificar que el fichero no está corrupto cuando al descomprimirlo recupere los bytes originales. Sigue descomprimiendo bytes hasta que consiga recuperar los que tenía el original. Si aún le quedan más de un byte (el relleno podría tener algunos bits más pero no más de 7 bits) los que obtiene no coinciden con los que tenía, y algo ha ido mal.
- **B (siempre 2 bytes)**: un short que contiene cuantos elementos hay en la tabla de frecuencias (máximo 256). Es decir, cuantos diferentes bytes tenía el fichero original. Desde 0 (si el fichero a comprimir estaba vacío) hasta 256 (todos los bytes posibles).
- **Tabla de frecuencias (B * 5 bytes, de 0 hasta como máximo 1280 bytes)**: la tabla de frecuencias nos va a permitir volver a reconstruir el árbol de huffman del fichero original. Cada entrada de la tabla tiene un byte (pesa 1 byte) y la frecuencia (la cantidad de veces que aparece en el fichero original) que es un int (4 bytes). Luego cada entrada pesa 5 bytes.
- **Mensaje comprimido**: los bytes con los datos codificados del mensaje original.

Como ves, cada fichero tiene que soportar cierto peso extra para poder deshacer el proceso de compresión. Como máximo el peso extra es 1280 + 8 + 2 + 4 + 3 = 1297 bytes como máximo. Para que el proceso de compresión sea efectivo, el fichero debe tener cierto peso, de lo contrario el ahorro que conseguimos lo perdemos por esta información.

Para generar el mensaje comprimido, debes volver a recorrer el fichero a comprimir desde el principio y sustituir cada byte por sus bits comprimidos. Yo te aconsejo que uses un String

para ir añadiendo bits y sigas el siguiente proceso:

```
String s ← cadena vacía
int longitud, aDisco;
Mientras (haya bytes en el fichero)
        int b ← siguiente byte del fichero
        s ← s + tablaCodigos[b]         // Añades al String, los bits del byte b
        longitud ← s.length()
        while ( longitud >= 8 ) {
                aDisco ← convertir a byte ( primeras 8 letras de s )
                s <- s.subcadena(7)    // El resto de s pendiente de convertir
                longitud -= 8;
                guardar en el fichero .huf el byte
        }
}
si longitud > 0 {  // Si quedan bits por guardar
        mientras longitud modulo 8 != 0  {
                s ← s + "0";
                longitud ++;
        }  // Rellenar el último byte
        aDisco ← convertir a byte ( s )
}
```

PASO 5 (1 PUNTO). HACER EL PROCESO DE DESCOMPRIMIR. Cuando el programa comienza, el botón de descomprimir está desactivado. Cuando abras un fichero, si la extensión es .huf, el programa activa el botón de descomprimir. Si se pulsa el botón, el código debe:

SUBPASO 1. Validar que los 3 primeros bytes son 'H', 'U', 'F', de no ser estos, lanza un mensaje de advertencia en un diálogo, desactiva el botón de descomprimir y sale del proceso.
SUBPASO 2. Leer extensión, N (total de bytes a descomprimir) y B (entradas en la tabla de frecuencias).
SUBPASO 3. Crear un array de 256 elementos enteros y reconstruir la tabla de frecuencias usada al comprimir.
SUBPASO 4. Con la tabla de frecuencias generar el árbol de huffman usado para comprimir.
SUBPASO 5. Comienza a leer los bytes del fichero y que los bits te guíen por el inóspito árbol hasta encontrar nodos hoja, momento en el cual ya sabes el byte que debes generar.

```
Long bytesDescomprimidos ← 0
nodoActual ← raíz del árbol huffman
MIENTRAS queden bytes por leer del fichero .huf AND bytesDecomprimidos < N HAZ
        b ← leer siguiente byte
        PARA i ← 1 HASTA 8 HAZ
                SI nodoActual no es una hoja
                        SI b AND 0x80 == 0  // Comprobar el bit
                        ENTONCES
                                nodoActual ← nodoActual.hijoIzquierdo()
                        SINO
                                nodoActual ← nodoActual.hijoDerecho();
                        FINSI
                SINO  // es una hoja
                        byteOriginal ← byte del nodo hoja nodoActual
                        Escribir byteOriginal en el fichero descomprimido.
                        BytesDescomprimidos ← bytesDescomprimidos + 1
                        nodoActual ← raíz del árbol huffman
                FINSI
```

```
                b ← b DESPLAZADO A LA IZQUIERDA 1  // o bien multiplicar por 2
        FINPARA
FINMIENTRAS
```

EJERCICIO 13. (+5 PUNTOS) ANALIZADOR EXPRESIONES.

Crea una plicación con interface gráfica que sea una calculadora capaz de leer una expresión algebraica (que use variables x además de valores enteros y doubles, operaciones (+) (-) (*) (/) (^) y funciones (sen, cos, tan, asen, acos, atan, exp, log, ln), expresiones como estas por ejemplo: 3, 4 * x, 1 / (x - 1) , 3 * x^(x + 1/ x) - 4 , x / (1 – sen(x)^2)

FUNCIONES f(x) = expresión con variables x.

Para ser capaces de extraer las operaciones que necesitamos realizar a partir de un texto que contiene una expresión, !!necesitamos un compilador!! Un traductor que traslade lo que representa la expresión a unas operaciones realizables por el programa.

- **HARDWARE VIRTUAL: ¿Qué usaremos para hacer los cálculos?**

  RAM VIRTUAL DE DATOS

  Los datos los dejaremos en un array dinámico de doubles llamado datos, reservando la primera posición para la variable x. El resto de posiciones serán los valores numéricos que aparezcan en la expresión. Nuestra RAM donde está los datos será este array. Por ejemplo: 3+2/x genera estas entradas en el array datos: [x][3][2]

  RAM VIRTUAL DE SENTENCIAS

  Para realizar los cálculos, nuestro programa estará almacenado en un array de sentencias llamado programa, usaremos una pila de operaciones y en vez de una CPU con registros, de forma que las operaciones se realizarán en orden infijo (la operación que se quiera ejecutar usará los datos de la pila), es como escribir "3 + 4" de la forma "+ 3 4" pero la operación estará en programa (la pila de sentencias) y los datos serán los del tope de la pila. Habrá sentencias que se aplican a dos datos y otras que se aplican a solo uno.

  ALU VIRTUAL

  Otra pila llamada alu será la encargada de ejecutar las operaciones. Las operaciones son:

```
OPERACION      DATO      DESCRIPCION                    PUEDE GENERAR ERROR
PUSH           pos       alu.push( datos[pos] )
SUMA           -----     alu.push( alu.pop() + alu.pop() )
CAMBIASIGNO    -----     alu.push( - alu.pop() )
RESTA          -----     alu.push( alu.pop() - alu.pop() )
MULTIPLICA     -----     alu.push( alu.pop()  * alu.pop() )
DIVIDE         -----     num ← alu.pop()  y den ← alu.pop()
                         SI den != 0
                         ENTONCES  alu.push( num / den )
                         SINO Warning → f(x) no definida para x
                              O BIEN alu.push( MAX_DOUBLE * signo(num) )
ELEVA          -----     alu.push( Math.pow( alu.pop(), alu.pop() ) )
SEN            -----     alu.push( Math.sin( alu.pop() ) )
COS            -----     alu.push( Math.cos( alu.pop() ) )
TAN            -----     alu.push( Math.tan( alu.pop() ) )
```

```
ASEN          -----        alu.push( Math.asin( alu.pop() ) )
ACOS          -----        alu.push( Math.acos( alu.pop() ) )
ATAN          -----        alu.push( Math.atan( alu.pop() ) )
EXP           -----        alu.push( Math.exp( alu.pop() ) )
LOG           -----        a = alu.pop();
                           SI a != 0 ENTONCES alu.push( Math.log10(a) )
                           SINO warning/Double.MAX_VALUE
LN            -----        a = alu.pop()
                           SI a > 0 ENTONCES alu.push( Math.log(a) )
                           SINO Warning / -MAX_VALOR
```

Por ejemplo: si escribimos la expresión "-3 + 4/x" nuestra máquina virtual quedará (el valor 0 en el programa significa la posición de la X, otros valores indican valores):

```
datos[]:    [x][3][4]
programa[]: [push 1][cambiaSigno][push 2][push 0][divide][suma]

Al ejecutarlo: (si a x le damos el valor 2: datos[0]= 2;)
sentencia 0:  pila ← [3]
sentencia 1:  pila ← [-3]
sentencia 2:  pila ← [-3][4]
sentencia 3:  pila = [-3][4][2]
sentencia 4:  pila = [-3][2]       // se ha sacado [4][X] y se mete [4] / [2]
sentencia 5:  pila = [-1]          // se ha sacado [-3][2] y se mete [-3] + [2]
```

Esto nos da idea de como ejecutaremos las sentencias (recorriendo el array desde el principio hasta el final). Para generar el programa que tenemos en el array a partir de cualquier expresión (recorrer el árbol sintáctico de la expresión en inorden: primero el hijo izquierdo, luego el hijo derecho, luego el nodo). Pero para generar este árbol sintáctico, hay que saber un poco como funcionan los lenguajes para ser capaces de "traducirlos" a esta estructura.

PASO 2 (1 PUNTO). LENGUAJE DE LAS EXPRESIONES Y EL ANALIZADOR LEXICO.

Los lenguajes que se utilizan en informática suelen ser formales (solo interesan ciertas estructuras linguísticas, las que no permiten definir cosas ambigüas). Un lenguaje formal se puede expresar mediante una gramática G.

Una gramática G consiste en 5 cosas G = (NT, T, P, S) que explico:

- **NT, conjunto de símbolos no terminales**. Son símbolos que permiten definir estados temporales. No los veremos escritos en los mensajes del lenguaje, pero ayudan a definir la estructura de las expresiones sintácticas que usa. Por ejemplo: PREDICADO ::= VERBO + COMPLEMENTO_VERBAL predicado sería un elemento no terminal de un lenguaje. Suelen escribirse en mayúsculas para diferenciarlos de los terminales.
- **T, conjunto de símbolos terminales**: son los símbolos que si vemos en los mensajes del lenguaje. Por ejemplo un predicado puede ser: volaba muy alto. Se indican en minúsculas.
- **P, conjunto de reglas de producción**: permiten indicar como se puede generar cualquier mensaje del lenguaje. Imagina una gramática con estas reglas:

  T1 → a T2
  T2 → b | b T1 b

  En esas regla T1 y T2 son no terminales (en mayúscula) o intermedios. Y "a" y "b" son terminales (o finales, y por eso están en minúscula). Estas reglas permiten generar

mensajes de tipo:

T1 → a T2 → ab
T1 → a T2 → ab T1 b → abaT2 b → ababb
T1 → a T2 → ab T1 b → abaT2 b → ababT1bb → ...

- **S, es el símbolo origen:** el símbolo no terminal a partir del cual se genera cualquier mensaje del lenguaje que define la gramática. Es como un punto de inicio.

Si definimos bien una gramática, podemos hacer un programa que vaya recorriendo los mensajes de su lenguaje, identificando sus partes y realizando alguna tarea, como por ejemplo ir generando otras sentencias que sean capaces de calcular el valor de esa expresión. Este programa se denomina **analizador sintáctico**. Hay muchos tipos, uno de ellos es de los más sencillos y se llama recursivo.

Para facilitarnos la tarea de reconocer los elementos de un mensaje, iremos identificando los elementos que vayamos encontrando en el mensaje. A estos elementos los llamamos **tokens**. En nuestro lenguaje de escribir expresiones necesitamos estos tokens:

El esqueleto del analizador léxico lo podemos definir así (falta completarlo):

```
class AnLex {

    public enum Token { SIGNOMAS, SIGNOMENOS, NUMERO, VARIABLE, SUMA, RESTA,
                        MULTIPLICA, DIVIDE, ELEVA, FUNCION, ABREPARENTESIS,
                        CIERRAPARENTESIS, INICIO, FINAL, ERROR }

    static final private String funciones ="sen,cos,tan,asen,acos,atan,exp,log,ln";

    private String expresion;        // La expresión a analizar
    private int longitud;            // cantidad de letras de la expresión
    private int posActual;           // Qué letra se va a analizar
    private Token tokenAnterior;     // Token que se ha encontrado antes
    public  String valorToken;       // letras que forman el token actual
    private String error;            // Mensaje de error léxico (token mal escrito)
    private char letra;              // última letra leída del mensaje

    public AnLex(String exp) {
        expresion = exp;
        longitud = (expresion == null)? 0: expresion.length();
        posActual = 0;
        tokenAnterior = Token.INICIO;
    }

    // quitar espacios
    public void quitarEspacios() {
        while(posActual < longitud && (letra=expresion.charAt(posActual)) == ' \n')
            posActual++;
    }

    public Token getToken() {
        if(tokenActual == Token.FINAL) return TokenFINAL;
        tokenAnterior = tokenActual;
        valorToken = "";
        quitarEspacios();
```

```
        if(posActual >= longitud) { tokenActual = TokenFinal; return Token.FINAL; }
        // según el valor de letra estaremos en un token u otro
        switch(letra) {
          case '-': // Si anterior es INICIO O ABREPARENTESIS SIGNOMENOS SINO RESTA
          case '+': // Si anterior es INICIO O ABREPARENTESIS SIGNOMAS SINO RESTA
          case 'x': case 'X': // VARIABLE
          case '*':    // MULTIPLICA
          case '/':    // DIVIDE
          case '^':    // ELEVA
          case '(':    // ABREPARÉNTESIS
          case ')':    // CIERRA PARÉNTESIS
          case '0': case '1': case '2': case '3': case '4': case '5': case '6':
          case '7': case '8': case '9': case '.': case ',':
                     boolean separador= (letra == ',' || letra == '.')? true:false;
                       // LOS NUMEROS PUEDEN SER:   1     12 123.   123.0  123,0 .45
                       // LOS NUMEROS NO PUEDEN SER:   1.2.3  2,3,4  2.3,4
          default:  // DEBE SER FUNCION, comprueba que es correcta SINO ERROR:
                    // sen, cos, tan, asen, acos, atan, exp, log, ln (mayús. o min)
          }
      }

      public String getValor() { // devuelve cadena del token actual }

      public String getError() { // devuelve último error léxico  }
}
```

Ejemplos de resultados: si cuando acabéis os hacéis un programa para ver lo que genera podéis probar estos ejemplos:

```
      public static void main(String[] args) {
            System.out.print("Teclee una expresión: ");
            String e = sc.nextLine();
            AnLex aL = new AnLex(e);
            Token t = Token.INICIO;
            while (t != Token.FINAL && t != Token.ERROR) {
                  System.out.print( t + " ");
                  t = aL.getToken();
            }
            System.out.println( t + " ");
            if(t == Token.ERROR) System.out.println("Error Lexico " + aL.getError() );
      }
```

```
Teclee una expresión: 3
INICIO NUMERO FINAL
```

```
Teclee una expresión: 3 * x
INICIO NUMERO MULTIPLICA VARIABLE FINAL
```

```
Teclee una expresión: -1 + cos(1/x)
INICIO SIGNOMENOS NUMERO SUMA FUNCION ABREPARENTESIS NUMERO DIVIDE VARIABLE CIERRAPARENTESIS FINAL
```

```
Teclee una expresión: +1 - 4 * x^2
INICIO NUMERO RESTA NUMERO MULTIPLICA VARIABLE ELEVA NUMERO FINAL
```

```
Teclee una expresión: 3+3/2
INICIO NUMERO SUMA NUMERO DIVIDE NUMERO FINAL
```

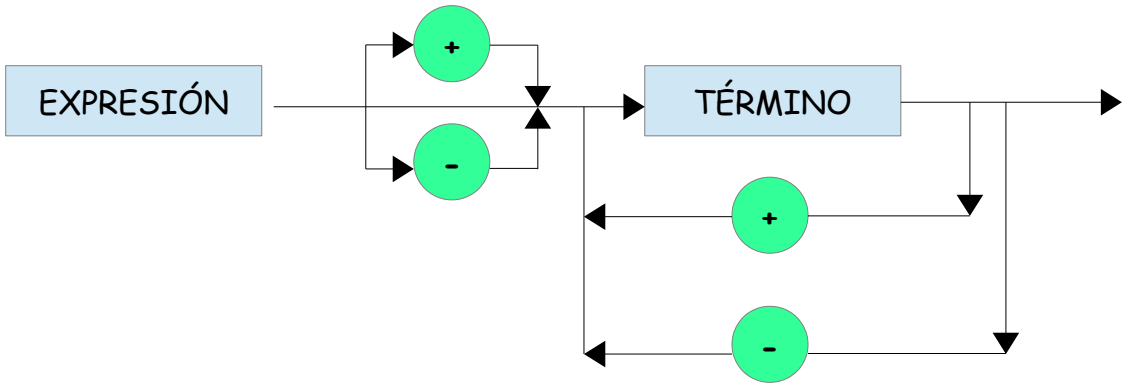
```
Teclee una expresión: (3+3)/2
INICIO ABREPARENTESIS NUMERO SUMA NUMERO CIERRAPARENTESIS DIVIDE NUMERO FINAL
```

PASO 3 (1 PUNTO). EL ÁRBOL SINTÁCTICO DE LAS EXPRESIONES:

Las reglas de producción de una gramática G, definen los mensajes bien construidos de un lenguaje. Las reglas se pueden expresar mediante diagramas de alambre (los símbolos no terminales en cuadrados y los terminales en círculos) o como reglas (inicio → combinación de terminales y no terminales). Por ejemplo el lenguaje de las expresiones que usa nuestra calculadora comienzan así porque E (de EXPRESIÓN) es el símbolo inicial de todos los mensajes del lenguaje (la | indica cada alternativa):

E → +T | -T | T | T + T1 | T – T1
T1 → T + T1 | T – T1 | F

En diagrama de alambres:



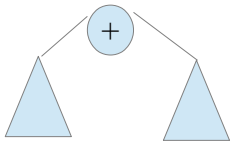
Los objetivos del analizador sintáctico son dos: **construir el árbol sintáctico** (árbol binario) que representa la estructura del mensaje (y que luego podremos procesar para generar las sentencias que sean capaces de hacer los cálculos) y por otro lado, detectar posibles errores sintácticos si el mensaje no sigue ninguna estructura capaz de producir las reglas de producción.

Para ello irá pidiendo tokens al analizador léxico y construyendo el árbol hasta que alcance el final del mensaje o encuentre errores. De forma genérica el analizador de esta regla comenzará así (más adelante te daré una versión más real de la misma):

```
public void analiza(){
    Token t= aL.getToken();
    expresion(Token t);   // Comenzamos a analizar por el símbolo inicial
}

public void expresion(Token t) {
   if( t == Token.SIGNOMENOS ) {    // El signo + se ignora porque no cambia nada
       // crear Token.SIGNOMENOS
       t = aL.getToken();
   }
   t= termino(t);
   while(t == Token.SUMA || t == Token.RESTA ) {
       // crear Token t
       t= termino(t);
   }
}
```

¿Cómo será un árbol sintáctico? En nuestro caso necesitamos que sea un árbol binario donde cada nodo tenga el token que representa y su valor, no todos, porque algunos nos ayudarán a definir la estructura del árbol (los paréntesis) y algunos podremos transformarlos (como el

signo menos delante de un número). Los operadores binarios (que trabajan con dos operandos o valores, estarán en la raíz de un árbol y sus valores serán sus hijos (uno izquierda y otro derecha).



Nuestra clase será este código donde he dejado todas las variables miembro públicos por eficiencia:

```
class ArbolSintactico {
    public Token t;
    public String valor;
    public ArbolSintactico hd, hi;

    public ArbolSintactico(Token token, String s, ArbolSintactico i, ArbolSintactico d) {
       t = token;
       valor = s;
       hi = i;
       hd = d;
    }

    // Otros elementos...
}
```

La clase AnSintac y un programa más realista, debe ocuparse además de seguir la estructura de las reglas de producción de la gramática, en ir construyendo el árbol sintáctico:

```
....
static public void main()
       Scanner sc = new Scanner( System.in );
       System.out.print("Teclee una expresión: ");
       String e = sc.nextLine();
       AnSintac aS = new AnSintac( e );
       ArbolSintactico arbol = aS.analiza();
}

class AnSintac {
    String expre;
    AnLex aL;
    ArbolSintactico arbol;
    String ultimoError;

    public AnSintac(String expresion) {
       expre = expresion;
       aL = new AnalizadorLexico( expre );
    }

    public ArbolSintactico analiza() {
        AnLex.Token t;
        t = aL.getToken();
        arbol = expresion(t); // Comenzamos a analizar por el símbolo inicial
        return arbol;
    }

    public ArbolSintactico expresion(Token t) {
       ArbolSintactico raiz, nuevo;
       if( t == Token.SIGNOMENOS ) {               // El signo + se ignora
           raiz = new ArbolSintactico(Token.SIGNOMENOS, null, null, null);
```

```
            t = aL.getToken();
        }
        if( raiz != null ) raiz.hi = termino(t);  else raiz = termino(t);
        t = aL.getToken();
        while( t == Token.SUMA || t == Token.RESTA ) {
            nuevo = new ArbolSintactico(t, 0, raiz, null);
            raiz = nuevo;
            t = aL.getToken();
            raiz.hd = termino(t);
            t = aL.getToken();
        }
        return raiz;
    }
```

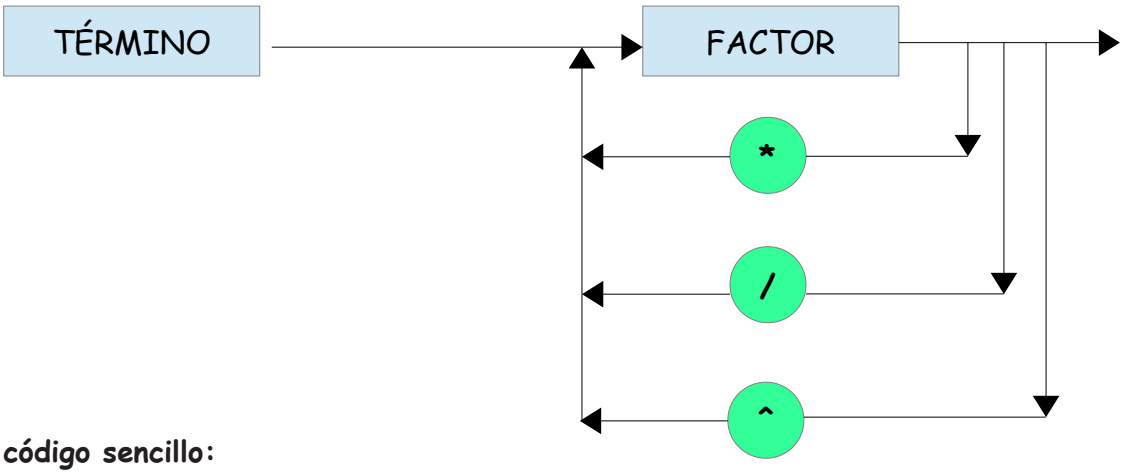
Para TERMINO (T), la regla genera elementos factores. Si vemos la regla en forma de expresiones, en diagrama de alambres y en programa sin entrar en detalles:

Reglas:

T → F | F * F1 | F / F1 | F ^ F1

F1 → T

Diagrama de alambres:



El código sencillo:

```
public arbolSintactico termino(Token t) {
   ArbolSintactico raiz = factor(t);
   t = aL.getToken();
   while( t == Token.MULTIPLICA || t == Token.DIVIDE || t == Token.ELEVA ) {
       // crear Token t
       ArbolSintactico nuevo = new ArbolSintactico( t, null, raiz, null );
       t = aL.getToken();
       nuevo.hd = factor(t);
       raiz = nuevo;
       t = aL.getToken();
   }
   return raiz;
}
```

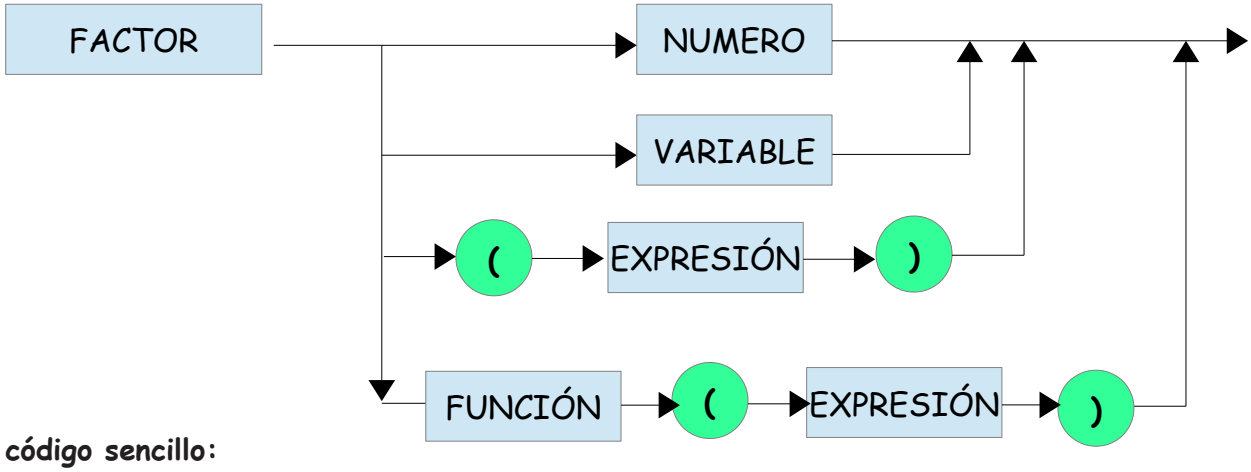
Y FACTOR (F) se expande a elementos más cercanos a símbolos terminales. Ahora además de construir el árbol habrá que recordar los valores de los tokens:

Reglas:

F → N | V | (E) | F(E)

// N es número, V es variable, F función, E expresión

Diagrama de alambres:

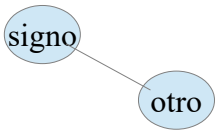


El código sencillo:

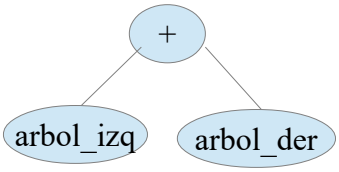
```
public ArbolSintactico factor(Token t) {
   ArbolSintactico raiz;
   if(t == NUMERO) return árbol número;
   if(t == VARIABLE) return árbol variable;
   if(t == ABREPARENTESIS) {
       t = aL.getToken();
       raiz = expresion(t);
       t = getToken();
       if( t != CIERRAPARENTESIS) devuelve error
       devuelve raiz;
   }
   if(t == FUNCION)  {
       raiz = new ArbolSintactico(t, t.valor, null, null);
       t = aL.getToken();
       if(t != ABREPARENTESIS) devuelve error;
       t = aL.getToken();
       raiz.hd = expresion(t);
       t = aL.getToken();
       if( t != CIERRAPARENTESIS) devuelve error
       devuelve raiz;
   }
   devuelve error
}
```

PASO 4 (1 PUNTO). GENERAR EL PROGRAMA A PARTIR DEL ÁRBOL SINTÁCTICO.

Una vez generado el árbol, queda traducir su estructura a una serie de sentencias (equivalentes a los bytecodes) que podamos ejecutar. Por ejemplo, rellenando una pila de sentencias (la pila será el programa que un trozo de código debe ejecutar):



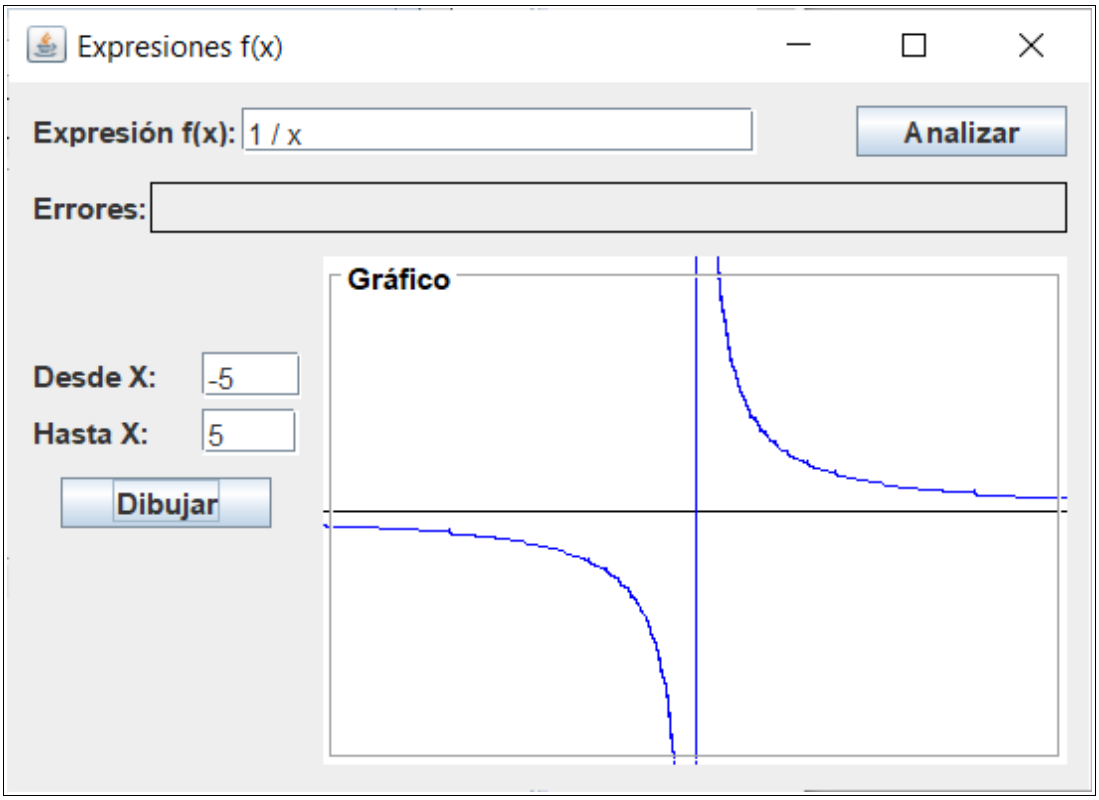

```
programa[sentencia++] = otro...
programa[sentencia++] = (cambiasigno)
```




```
programa[sentencia++] = arbol_izq
programa[sentencia++] = arbol_der
programa[sentencia++]= (suma, 0)
```

Es decir, salvo si el nodo es un cambio de signo (que la expresión estará siempre a la derecha, o la izquierda, lo que tu decidas) el resto de sentencias se recorren en inorden: primero el subárbol de la izquierda, luego el subárbol de la derecha y por último se genera el nodo de la sentencia.
Tras ejecutar el método generaPrograma(ArbolSintactico, programa, datos, cpu) todo quedará listo para ejecutar cualquier expresión válida sin más que ir recorriendo el programa desde la primera sentencia hasta la última y si hay variables en la expresión, sustituir el valor que van a tomar en el array de datos. Observa que datos y sentencias nunca cambian en cada ejecución salvo el valor de x (en la posición 0 de los datos).

LA CALCULADORA (1 PUNTO): ahora hay que unir todo esto y darle una interfaz. Crea cuadros de texto para escribir expresiones, límite inferior de X donde comenzar a dibujar, límite superior de X y un botón para iniciar el dibujo. Un panel donde dibujar el gráfico (los ejes horizontal (X) y vertical (Y). Puedes hacer que el Y sea del mismo tamaño que el X o escalarlo, dibujar marcas de valores, etc.. Queda a tu elección.


Expresiones f(x)
Expresión f(x):
1 / x
Analizar
Errores:
Gráfico
Desde X:
-5
Hasta X:
5
Dibujar